МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Фізико-технічний інститут

ЗВІТ

з комп’ютерного практикуму №4

з дисципліни «Криптографія»

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Виконали: студентки групи ФБ-43
Горбонос Олександра
Дрєпіна Анастасія
Викладач: Селюх П.

Київ – 2026

1 Мета роботи

Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

2 Постановка задачі та варіант завдання

У методичці та в папці `tasks/lab4` репозиторію курсу варіантів немає: усі параметри обираються випадково. Потрібно:

- написати функцію пошуку випадкового простого числа заданої довжини на основі вбудованого генератора псевдовипадкових чисел і власноруч реалізованого тесту Міллера-Рабіна з попередніми пробними діленнями;
- згенерувати дві пари простих чисел p, q і p_1, q_1 довжиною щонайменше 256 біт так, щоб $pq \le p_1 q_1$;
- написати функцію генерації ключів RSA і побудувати схеми для абонентів A і B;
- реалізувати окремими процедурами шифрування, розшифрування, підпис і перевірку підпису, передаючи їм лише необхідні ключові дані;
- реалізувати протокол конфіденційного розсилання ключів із підтвердженням справжності відправника;
- перевірити кожну операцію взаємодією з тестовим середовищем.

3 Теоретичні відомості

Піднесення до степеня. За схемою Горнера показник α записується у двійковій формі, а значення обчислюється зліва направо: $y := y^2 \bmod m$, і якщо біт $\alpha_i = 1$, то $y := y \cdot x \bmod m$. Потрібно не більше $k - 1$ піднесень у квадрат і $k - 1$ множень, де k — довжина α у бітах.

Пробні ділення. За ознакою подільності Паскаля для $N = \Sigma a_i \cdot B^i$ виконується $N \equiv \Sigma a_i \cdot r_i \pmod m$, де $r_0 = 1$, $r_{i+1} = r_i \cdot B \bmod m$. Це дозволяє швидко відкинути більшість складених чисел.

Тест Міллера-Рабіна. Нехай $p - 1 = d \cdot 2^s$, d непарне. Число p сильно псевдопросте за основою x, якщо $\gcd(x, p) = 1$ і $x^d \equiv \pm 1 \pmod p$ або $x^{d \cdot 2^r} \equiv -1 \pmod p$ для деякого $0 < r < s$. Для складеного p ймовірність пройти k випадкових основ не перевищує 4^{-k}.

RSA. Абонент обирає прості p, q, обчислює $n = pq$, $\varphi(n) = (p - 1)(q - 1)$, обирає e таке, що $\gcd(e, \varphi(n)) = 1$, і знаходить d: $ed \equiv 1 \pmod{\varphi(n)}$. Відкритий ключ — (n, e), секретний — (d, p, q).

$$C = M^e \bmod n, \quad M = C^d \bmod n, \quad S = M^d \bmod n, \quad \text{перевірка: } M = S^e \bmod n$$

Протокол розсилання ключа. A має ключі (e, n), d і відкритий ключ B (e_1, n_1), причому обов'язково $n_1 \ge n$. A надсилає (k_1, S_1), де:

$$S = k^d \bmod n, \quad k_1 = k^{e_1} \bmod n_1, \quad S_1 = S^{e_1} \bmod n_1$$

B обчислює $k = k_1^{d_1} \bmod n_1$, $S = S_1^{d_1} \bmod n_1$ і перевіряє $k = S^e \bmod n$.

4 Хід роботи

4.1 Програмна реалізація

Програму написано мовою Python. Усі математичні функції реалізовано власноруч у модулі `code/rsa.py`; вбудовану функцію `pow` і бібліотеку `sympy` використано лише у файлі `verify.py` для незалежної перевірки результатів. Інтерфейси високорівневих функцій наведено в таблиці 1.

Таблиця 1 — Високорівневі функції та їхні вхідні дані

Функція	**Вхід**	**Результат**
GenerateKeyPair(p, q, e)	прості p, q; e = $2^{16} + 1$	(n, e) і (d, p, q)
Encrypt(M, e, n)	M, відкритий ключ адресата	C
Decrypt(C, d, n)	C, секретний d, модуль n	M
Sign(M, d, n)	M, секретний d, модуль n	S
Verify(M, S, e, n)	M, S, відкритий ключ підписувача	так / ні
SendKey(k, d, n, e1, n1)	k, ключі відправника d, n; відкритий ключ одержувача	(k_1, S_1)
ReceiveKey(k1, S1, d1, n1, e, n)	(k_1, S_1), ключі одержувача d_1, n_1; відкритий ключ відправника	(k, ознака прийняття)

Датчик випадкових чисел — вбудований модуль `random`, як вимагає методичка. Для відтворюваності зафіксовано початкове значення `seed` (не з методички), щоб повторний запуск давав ті самі числа. Для реальних ключів такий датчик не підходить, бо він не є криптографічно стійким.

4.2 Генерація простих чисел

Пошук виконано за схемою з розділу 5 методички: обирається випадкове x з інтервалу [2^{255}, $2^{256} - 1$], знаходиться найменше непарне $m_0 \geq x$, і перевіряються числа $m_0 + 2i$. Кожен кандидат спершу проходить пробні ділення на всі прості менші за 1000 (ознака Паскаля, B = 2^{16}), а потім 40 раундів тесту Міллера-Рабіна. Межа 1000 і кількість раундів 40 обрані нами (не з методички): імовірність помилки не перевищує 4^{-40}.

Таблиця 2 — Вибрані прості числа (шістнадцятковий запис)

Число	**Значення**
p	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD9CF`
q	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 342989A7`
p_1	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E3D`
q_1	`B0EC8513 0333A36A AFB930FC CF26DD01 03CD3BAB FEB950F5 81EA77F8 12AA4BBB`

Таблиця 3 — Кандидати, що не пройшли перевірку

Число	Перевірено	Відкинуто діленням	Відкинуто Міллером-Рабіном	Біт
p	66	53	12	256
q	110	95	14	256
p_1	221	190	30	256
q_1	39	33	5	256
Разом	436	371	61	—

Усього перевірено 436 кандидатів. Пробні ділення відкинули 371 (85,1 %), тому дорогий тест Міллера-Рабіна запускався лише для 65 чисел. Це узгоджується з твердженням методички, що пробні ділення швидко відсікають більшість складених чисел. Повний перелік відкинутих кандидатів наведено в додатку А.

Вийшло $pq \le p_1 q_1$, тому міняти пари місцями не довелося. Простоту всіх чотирьох чисел і складеність усіх відкинутих кандидатів додатково підтверджено функцією `sympy.isprime` (файл `verify.py`).

4.3 Проміжні результати тесту Міллера-Рабіна

Для числа p_1 наведено раунд №3 (усього 40). Цей раунд обрано тому, що в ньому видно обидва кроки тесту; у частині інших раундів уже $x^d \bmod p = \pm 1$.

Таблиця 4 — Проміжні результати для простого числа

Величина	Значення
p	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E3D`
d	`24318033 010EE578 0542B1EB E7440F5A C155EA95 72B0C7B0 DEF23511 4754278F`
s	`2`
Основа x	`19B5EB5 874D0264 4D687D1F 18E17CBD C38055CF 4F85C826 B8A8117E DA331597`
$x^d \bmod p$	`142887AB EA15C6B3 7A801091 FCD0F230 EABBE71B 33478F40 E2ABC810 163E5E9B`
$x^{2d} \bmod p$ (r = 1)	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E3C`
p − 1 для порівняння	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E3C`

На кроці r = 1 отримано $x^{2d} \equiv -1 \pmod p$, отже p сильно псевдопросте за цією основою.

Для порівняння (не з методички): кандидат `852D78B7 9761D2B3`… пройшов пробні ділення, але за основою `5022E8F4 FA568206`… при s = 2 жодне зі значень не дорівнює ±1, тому він складений.

4.4 Ключі RSA абонентів A і B

Таблиця 5 — Параметри абонентів

Параметр	Значення
n (A)	4731D82B 99BCC2BC B8B47824 A0E45D91 934ED3D3 EB3E1BB6 4EB10543 E754AAAE 012D273E 46B7359B BFD1EE8B D0541D08 5EDBC3EF 3B535021 6092B99C 19AFDD09
e (A)	10001
d (A)	3A0A54A5 16EC6005 E2BE9DC3 9EB70DE1 042A9B5E 84C5596A 42B4E609 F9A9788B 0E9E15EB 07276DD2 014CE7D2 C4935C04 6E5C7396 BB8DF15C 9C1F052C 362CBC11
n_1 (B)	640DE256 9DE8712E 72E434EB 4AF0FAA7 2045C6EE B9D7CF71 49DDCE50 BDAE3923 FC595D44 1C1C2A5D 95A95379 262A243B 45489E91 B2EABFBC 544216D4 66C1758F
e_1 (B)	10001
d_1 (B)	17877C56 CA43C37A 1CE2B5DF 2F7C8435 914D577F 048D4FE7 606B9CFD 37DE24E9 99E36A7C 06E4B4BC AC0B83D2 E957D252 BDDDD8A5 5F4BE5DD 2319E962 AE2AFAE1

Модулі мають 511 біт, бо добуток двох 256-бітних чисел має 511 або 512 біт. Умова $n \le n_1$ виконується.

Для абонента A також обчислено φ(n):

4731D82B 99BCC2BC B8B47824 A0E45D91 934ED3D3 EB3E1BB6 4EB10543 E754AAAC F325256F BE284F8B B2558FF4 40CF7495 ED7D698F F39A5430 70858E04 27E87994

Таблиця 6 — Перевірка співвідношень для пари ключів абонента A

Умова	Виконується
$n = p \cdot q$	так
$\varphi(n) = (p-1)(q-1)$	так
$p \neq q$	так
$2 \le e \le \varphi(n) - 1$	так
$\gcd(e, \varphi(n)) = 1$	так
$e \cdot d \bmod \varphi(n) = 1$	так
$0 < d < \varphi(n)$	так
$(M^e)^d \bmod n = M$ для тестового M	так

Такі самі перевірки виконано для абонента B: усі виконуються.

4.5 Шифрування, розшифрування і підпис

Таблиця 7 — Приклади для абонента A

Величина	Значення
M	2603CE54 DE391DB0 6C761A48 2B7A9742 A21AD572 9D69934A 690106E6 1927BEDC 9AC80625 D7D84A9D FDEA186C 26A9B1A3 C9DF2D97 AE720219 CCDD67F2 20F3879E
$C = M^e \bmod n$	17887C0 556C6096 2718B51D 3ABE95B0 2F85325A 0E0C51C7 447EFDAB EA775F81 2B47A61A 2C0C9BAE 1DDFE0B8 A570719C 253F167E 26E06693 1E17E1A1 30C7907F
Decrypt(C)	2603CE54 DE391DB0 6C761A48 2B7A9742 A21AD572 9D69934A 690106E6 1927BEDC 9AC80625 D7D84A9D FDEA186C 26A9B1A3 C9DF2D97 AE720219 CCDD67F2 20F3879E
$S = M^d \bmod n$	39A8EF05 525746FB 3152CCA6 42A2C038 734D80CF 436221F7 4152FD92 0CB182AB F36C4849 79C22A0A C5299F91 C1BE00E1 479E463F 4163E57B A5CC2710 43A941C0

Таблиця 8 — Приклади для абонента B

Величина	Значення
M	`47A89BCE D5823127 29B8A4F9 7ADA6D6E 2F9BCDD5 F4C2BE73 E052EE1A 906B0ECB 7A428F45 114AB0C8 D2183517 7C410181 0C55F5D5 DEFE2ECC 565FBD13 F3CBF2EB`
$C = M^{e_1} \bmod n_1$	`35098A1 47E08941 4455682B 84CF8C78 D7D50183 501A06B7 DE95BCB4 1E8C60BF 4D061A0B 1EE1A449 1692490A 9DB18FAF A9AD048B 5D792A55 EE56CCF9 E705D466`
Decrypt(C)	`47A89BCE D5823127 29B8A4F9 7ADA6D6E 2F9BCDD5 F4C2BE73 E052EE1A 906B0ECB 7A428F45 114AB0C8 D2183517 7C410181 0C55F5D5 DEFE2ECC 565FBD13 F3CBF2EB`
$S = M^{d_1} \bmod n_1$	`5BEBA954 C429F2C7 1AFC7B5A 4A3C12C4 B4461BE4 B5BEBC0B E55E0B9C 313AE9AA C22F3708 35CEACE1 50519588 804BB6F0 A2DC2964 39772C02 98FAC9D1 06E27DD9`

Для обох абонентів Decrypt(C) = M, а Verify(M, S) повертає «так». Для M + 1 або S + 1 перевірка повертає «ні». Відповідно до методички підпис обчислено безпосередньо над M, без геш-функції; на практиці підписують H(M).

4.6 Протокол конфіденційного розсилання ключа

A передає B випадковий ключ $0 < k < n$. Послідовність значень наведено в таблиці 9.

Таблиця 9 — Одне виконання протоколу

Хто	Обчислення	Значення	Ключ / властивість
A	k — випадкове	`1EFFEE2D 00A6CE70 AE0189E4 E3E13350 56C71B74 222506A8 941FA932 4F25E2EC 2BD8B2BD 2325354A F2B68F64 45CB8FAF 370EDF19 1CF95F0B E7BB8BAD 06607CA5`	—
A	$S = k^d \bmod n$	`7FCA20A 9BF2CEFA 8B6DDB67 83426628 00E9C5EF 34F1621D 3B105D17 974F5124 58EB0262 33E116E9 80F1D647 CC820EC9 5974E4D2 A5522404 600A4774 A80FA479`	секретний d абонента A: автентичність
A → B	$k_1 = k^{e_1} \bmod n_1$	`5266B598 D0CA5638 66BC7290 B45A9B30 8DBA3033 412F6CEC 39416B0C 555A903D 9142EE38 AA5ED849 94DFA9B9 3581D482 C2EBADCE 3E928FB3 6DE6CD2B AD7C5761`	відкритий (e_1, n_1): конфіденційність
A → B	$S_1 = S^{e_1} \bmod n_1$	`31EA7160 91519AB2 4FE5CF08 5330315B A7FEFD81 D0F22DF3 A4BF127A 4176D3B3 C59DFBBA D66DE955 A33D6644 67E9C87B 562A51F2 A92937C8 CEC14773 855FB9E4`	відкритий (e_1, n_1): конфіденційність
B	$k = k_1^{d_1} \bmod n_1$	`1EFFEE2D 00A6CE70 AE0189E4 E3E13350 56C71B74 222506A8 941FA932 4F25E2EC 2BD8B2BD 2325354A F2B68F64 45CB8FAF 370EDF19 1CF95F0B E7BB8BAD 06607CA5`	секретний d_1
B	$S = S_1^{d_1} \bmod n_1$	`7FCA20A 9BF2CEFA 8B6DDB67 83426628 00E9C5EF 34F1621D 3B105D17 974F5124 58EB0262 33E116E9 80F1D647 CC820EC9 5974E4D2 A5522404 600A4774 A80FA479`	секретний d_1
B	$S^e \bmod n = k$?	`так`	відкритий (e, n) абонента A: автентичність

Конфіденційність забезпечує шифрування на відкритому ключі B: лише власник d_1 може отримати k. **Підтвердження справжності** забезпечує підпис на секретному d абонента A: перевірка відкритим ключем A проходить лише для підпису, створеного власником d.

Таблиця 10 — Негативні сценарії протоколу (не з методички)

Сценарій	**Результат**
B отримав випадковий підпис замість справжнього	відхилено
змінено k_1 під час передавання	відхилено
для перевірки взято чужий відкритий ключ відправника	відхилено
SendKey викликано при $n_1 < n$	помилка, ключ не надіслано

4.7 Перевірка взаємодією з тестовим середовищем

Перевірку виконано через вебінтерфейс `asymcryptwebservice.appspot.com/?section=rsa` в одній сесії браузера. Числа передаються у шістнадцятковому записі, тип повідомлення — Bytes. Поля інтерфейсу обрізають довгі значення, тому повний протокол обміну збережено у файлі `data/server_session.json`. Файл `task5_server.py` перераховує локальну частину кожної операції та звіряє її з відповідями сервера.

Таблиця 11 — Результати перевірки

Операція	**Збіг**
Encrypt локально → Decrypt на сервері	так
Encrypt на сервері → Decrypt локально	так
Sign на сервері → Verify локально	так
Sign локально → Verify на сервері	так
SendKey на сервері → ReceiveKey локально	так
SendKey локально → ReceiveKey на сервері	так
Локальний Verify відхиляє підпис зі зміненим останнім бітом	так

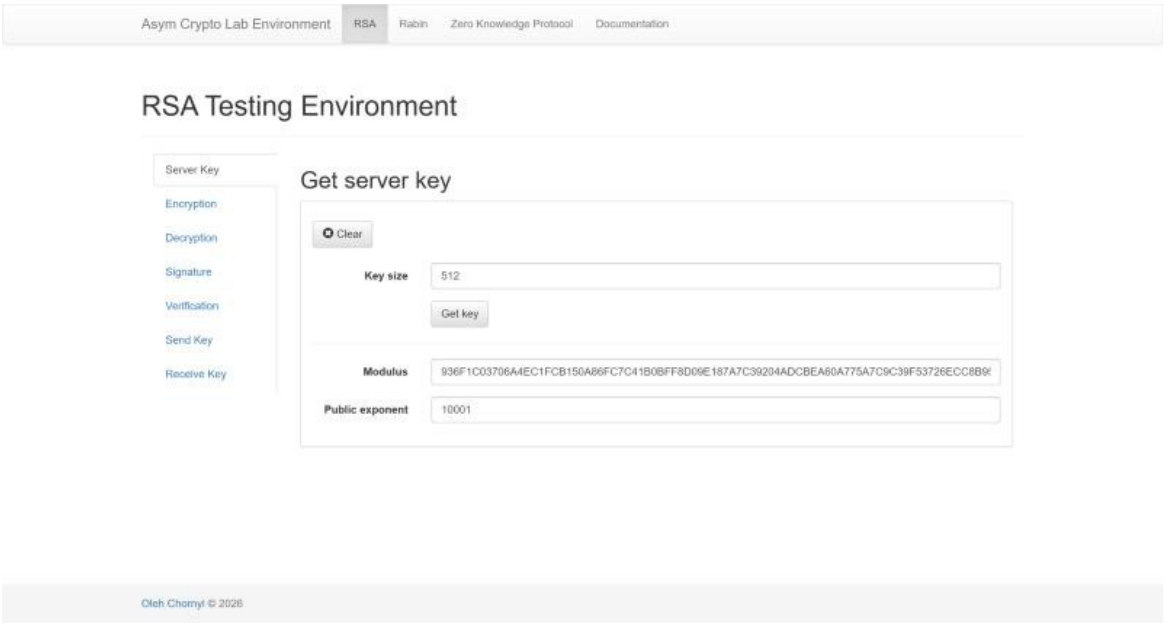


Рисунок 1 — Отримання 512-бітного ключа сервера

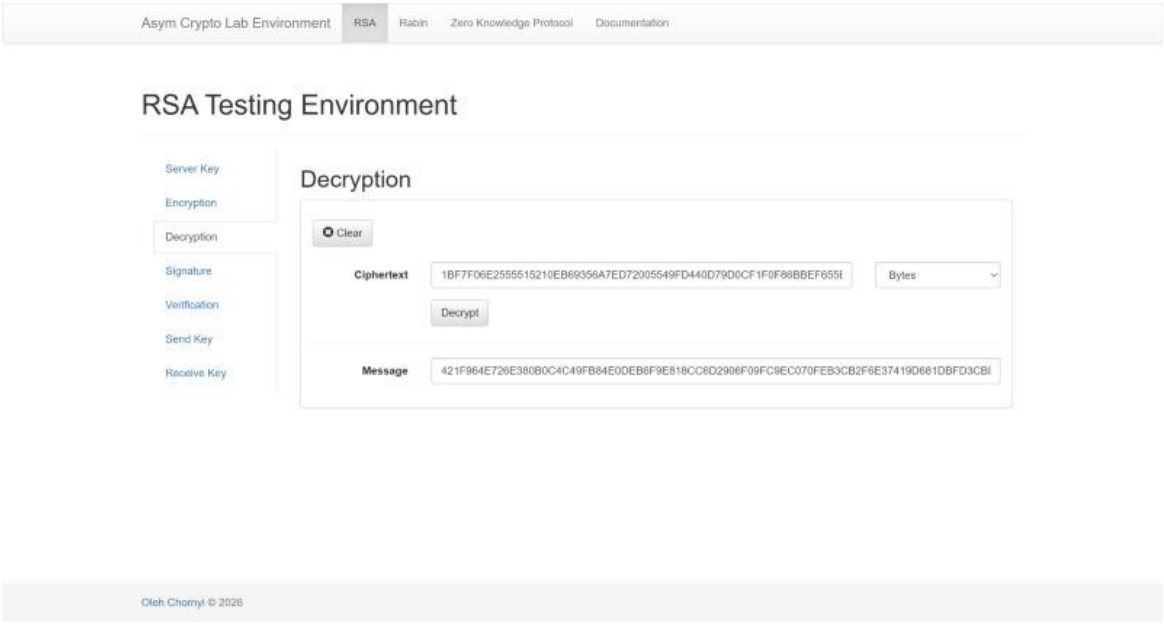


Рисунок 2 — Сервер розшифрував повідомлення, зашифроване нашою програмою

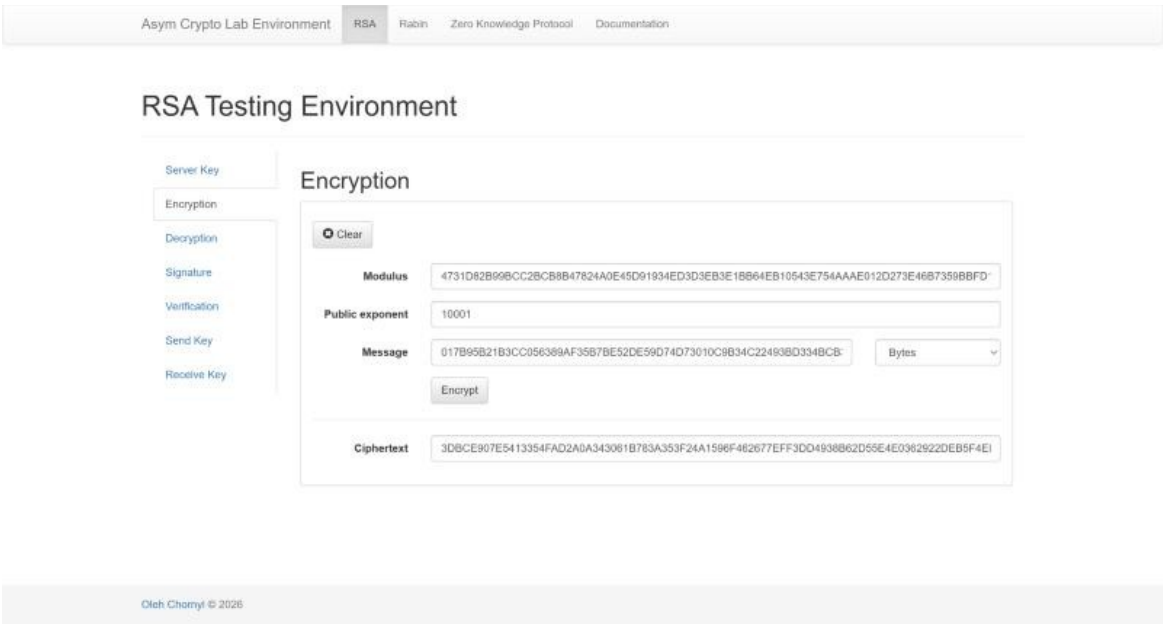


Рисунок 3 — Сервер зашифрував повідомлення для ключа абонента A; результат розшифровано локально

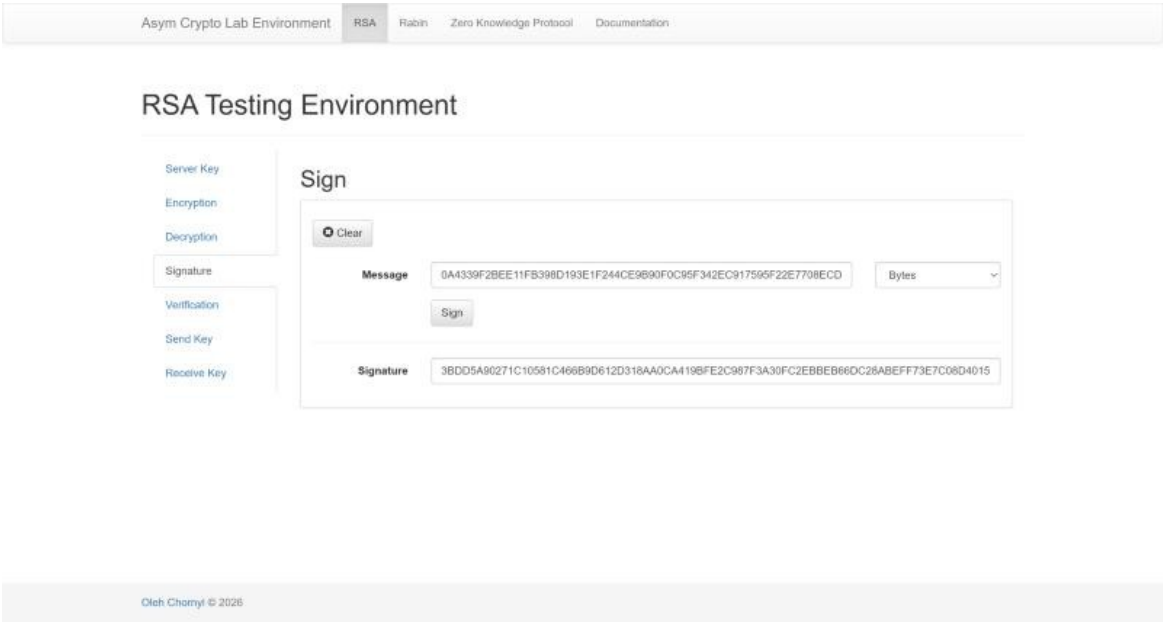


Рисунок 4 — Підпис сервера; результат перевірено локально функцією Verify

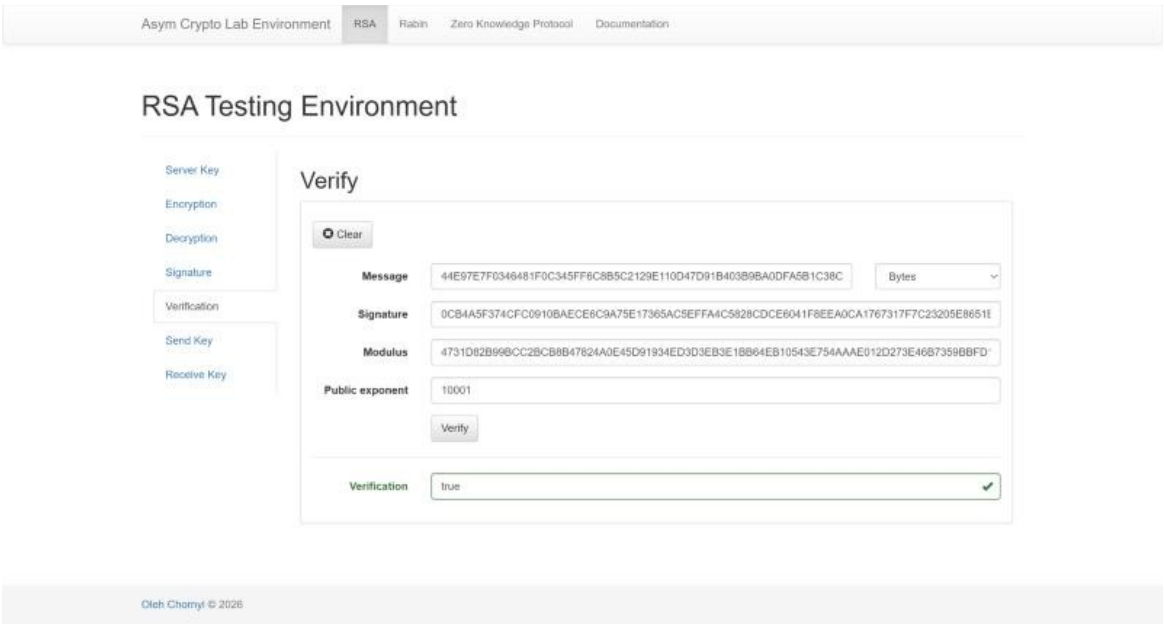


Рисунок 5 — Сервер підтвердив підпис, створений нашою функцією Sign

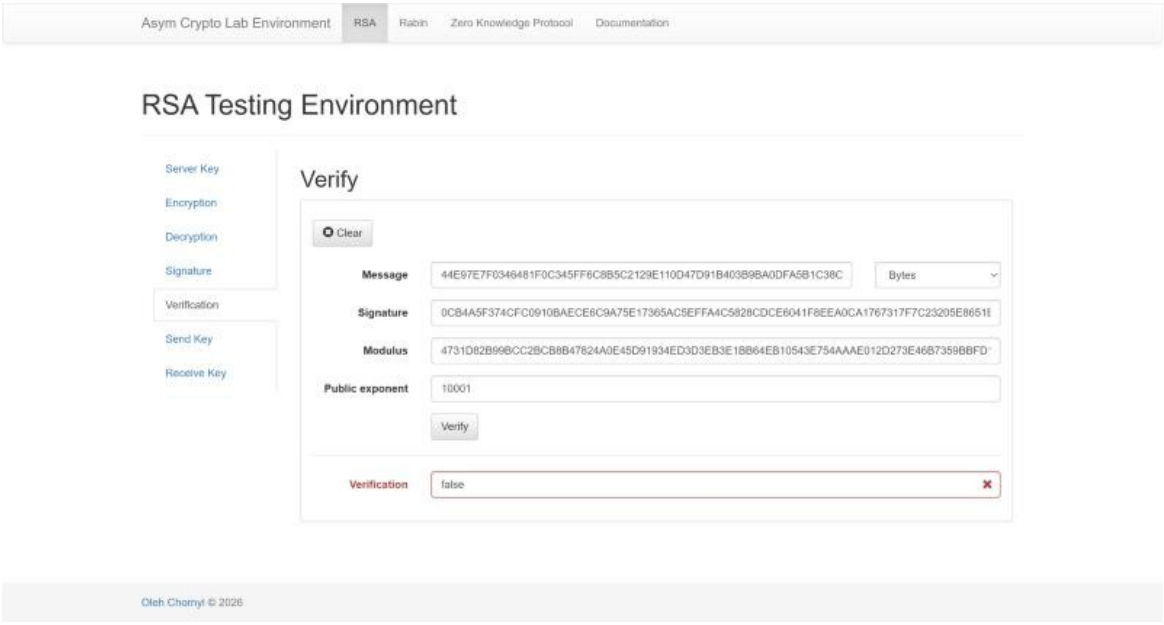


Рисунок 6 — Сервер відхилив той самий підпис зі зміненим останнім бітом

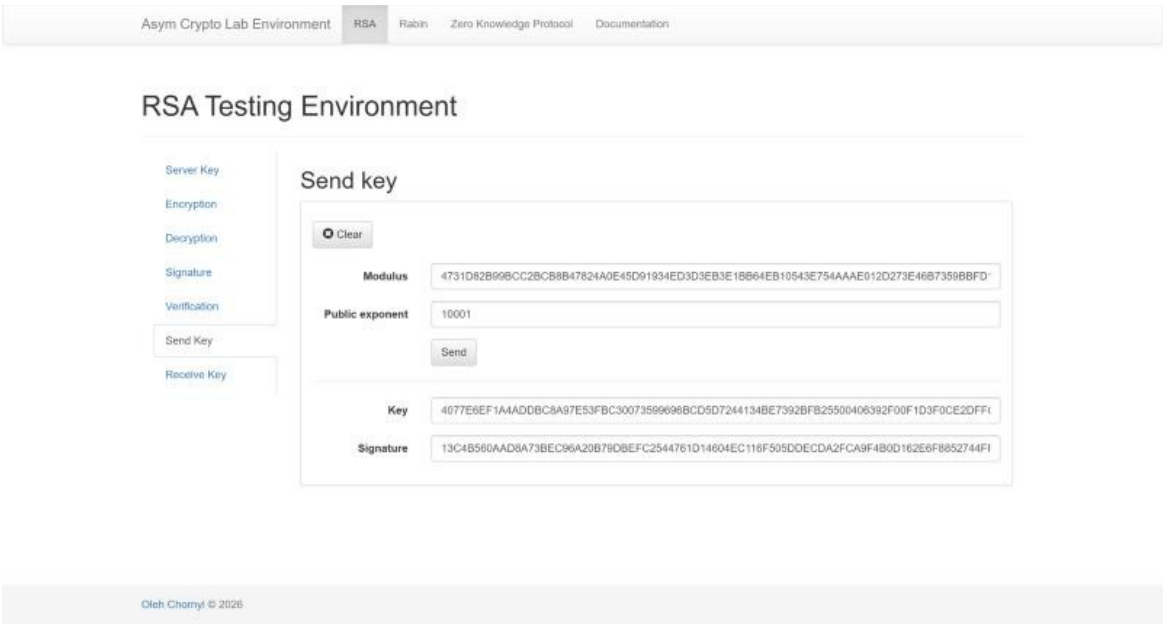


Рисунок 7 — Сервер із 256-бітним ключем надіслав ключ абоненту A; результат прийнято функцією ReceiveKey

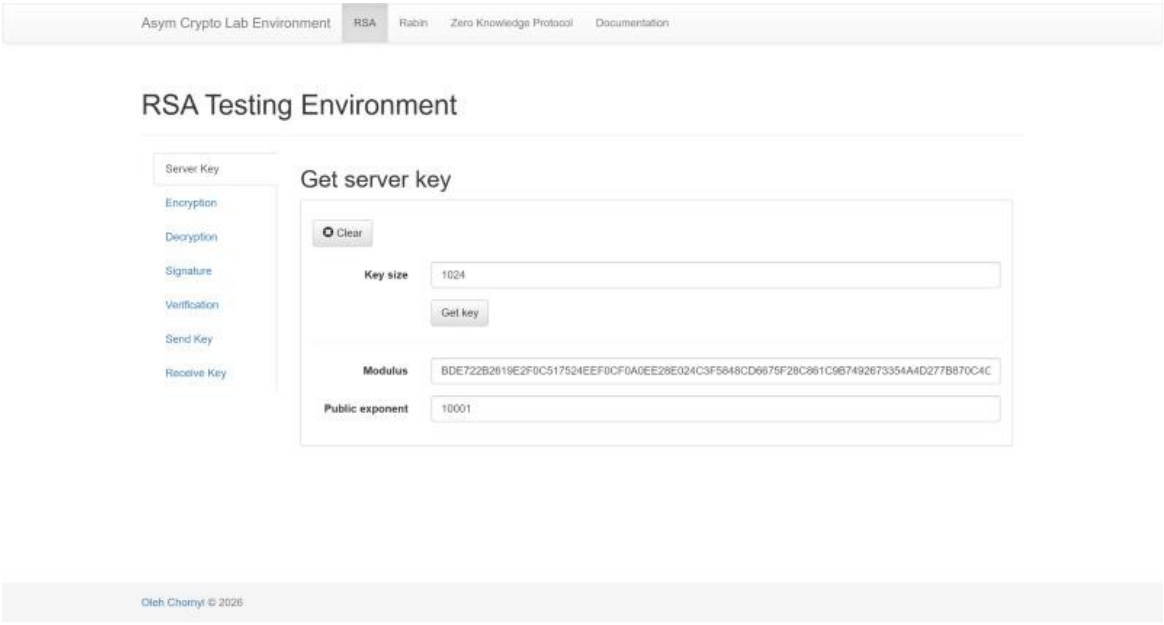


Рисунок 8 — Отримання 1024-бітного ключа сервера

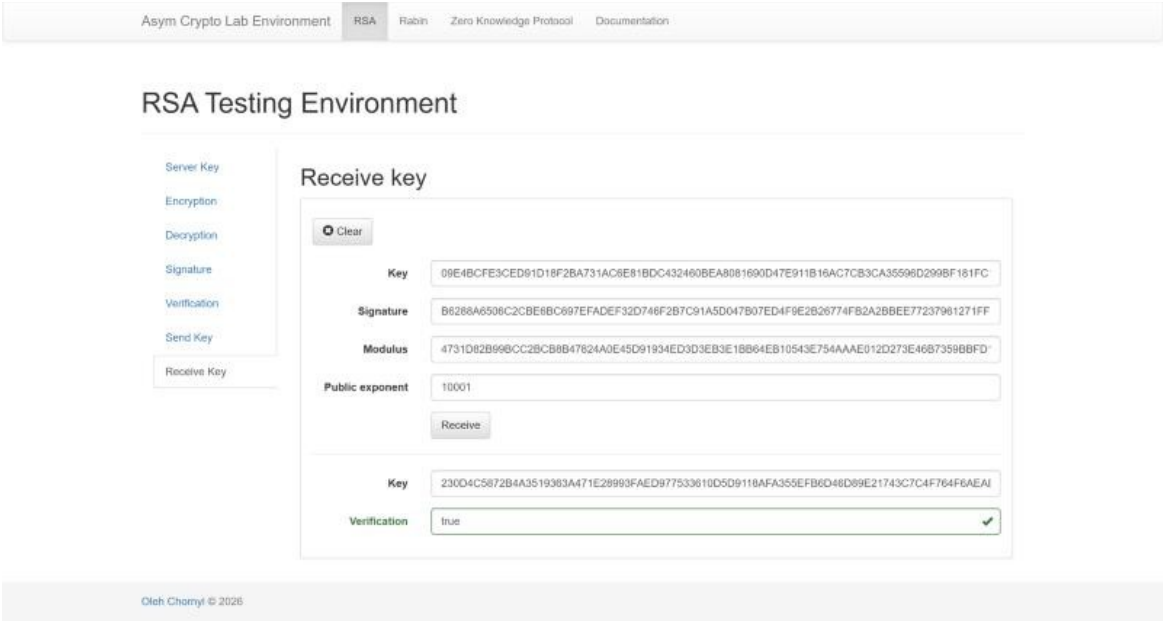


Рисунок 9 — Сервер прийняв ключ, надісланий функцією SendKey, і підтвердив підпис

4.8 Труднощі та шляхи їх розв’язання

- **Умова $n_1 \geq n$ під час обміну з сервером.** Наш модуль має 511 біт, а модуль сервера для 512-бітного ключа виявився більшим за нього. Тому для приймання ключа від сервера ми запросили менший ключ на 256 біт, а для надсилання ключа серверу — більший на 1024 біти.
- **Поведінка сервера при порушенні цієї умови.** Спершу ми викликали Send Key при 512-бітному ключі сервера, тобто при n_сервера > n_A. Сервер не відмовив, і перевірка пройшла. Це випадковість: значення підпису цього разу виявилося меншим за n_A. Якби воно було більшим, після зведення за модулем n_A підпис було б втрачено. Тому як результат ми враховуємо лише запуск із 256-бітним ключем.
- **Ключ сервера змінюється в межах сесії.** Кожен запит Get key замінює поточний ключ сервера, тому значення k_1 і S_1 обчислювались лише після

отримання потрібного ключа. Операції шифрування на сервері й перевірки використовують переданий нами відкритий ключ, а Decrypt, Sign, Send Key і Receive Key — поточний ключ сервера.

– **Діапазон повідомлень.** M має бути меншим і за наш модуль, і за модуль сервера, тому під час обміну числа обиралися менші за min(n_A, n_сервера).
– **Формат чисел.** Сервер працює із шістнадцятковими рядками, тому числа доповнюються нулем до парної довжини. Порівняння значень виконується як порівняння цілих чисел.
– **Спостереження.** Ключ k, згенерований сервером, має значно меншу довжину за модуль (59 біт для 256-бітного ключа). Для перевірки протоколу це не заважає.

5 Відповіді на контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і навіщо потрібна кожна з них? Спершу відбираються лише непарні кандидати $m_0 + 2i$, бо парні числа більші за 2 складені. Далі йдуть пробні ділення на прості менші за 1000: це дешевий спосіб відкинути більшість складених чисел (у нас 85,1 %). Потім виконується 40 раундів тесту Міллера-Рабіна: у кожному перевіряється $\gcd(x, p) = 1$ і сильна псевдопростота за випадковою основою. Цей тест виявляє складені числа, які не мають малих дільників, зокрема числа Кармайкла. Імовірність помилки не перевищує 4^{-40}. Також перевіряється довжина числа (задана інтервалом) і те, що $p \neq q$.

2. Які умови повинні виконуватися для параметрів пари ключів і як це перевірено? Потрібно: p і q прості та різні, $n = pq$, $\varphi(n) = (p - 1)(q - 1)$, $2 \le e \le \varphi(n) - 1$, $\gcd(e, \varphi(n)) = 1$, $ed \equiv 1 \pmod{\varphi(n)}$, $0 < d < \varphi(n)$. Для протоколу ще потрібно $n \le n_1$. Функція `GenerateKeyPair` викликає помилку, якщо $p = q$ або e необоротне. Решту перевіряє `task2_keys.py` (таблиця 6), а `verify.py` окремо перераховує ці значення.

3. Які дані необхідні для Encrypt(), Decrypt(), Sign() та Verify()? Чому не потрібні всі параметри RSA? Encrypt: M, e, n. Decrypt: C, d, n. Sign: M, d, n. Verify: M, S, e, n. Для обчислення потрібні лише показник і модуль. Шифрувати й перевіряти підпис може будь-хто, тому ці операції не мають отримувати секретні значення. Передавання p, q чи $\varphi(n)$ там, де вони не потрібні, лише збільшило б ризик витоку.

4. Які властивості забезпечуються на різних етапах протоколу? Формування $S = k^d \bmod n$ на секретному ключі відправника дає підтвердження справжності. Шифрування k_1 і S_1 відкритим ключем одержувача дає конфіденційність: k і S може отримати лише власник d_1. Перевірка $k = S^e \bmod n$ відкритим ключем відправника підтверджує, що k справді підписав власник d.

5. Які перевірки повинен виконати одержувач перш ніж вважати ключ коректним і справжнім? У нашій реалізації `ReceiveKey` перевіряє, що k_1 і S_1 лежать у $[0, n_1)$, що $0 < k < n$, що $0 \le S < n$, і що $S^e \bmod n = k$. Також одержувач має використовувати справжній відкритий ключ відправника. Протокол сам цього не гарантує, тому на практиці автентичність відкритого ключа підтверджують окремо (не з методички).

6 Висновки

У роботі реалізовано генерацію простих чисел, RSA, електронний підпис і протокол розсилання ключів. Для пошуку чотирьох 256-бітних простих чисел перевірено 436 кандидатів, причому пробні ділення відкинули 85,1 % із них, а тест Міллера-Рабіна — ще 61. Отже, попередні ділення суттєво зменшують кількість викликів дорогого тесту.

Шифрування, розшифрування, підпис і протокол правильно працюють для обох абонентів, а змінені дані та підписи відхиляються. Обмін із тестовим середовищем пройшов успішно в обох напрямках для всіх операцій, тож наша реалізація сумісна зі сторонньою. На практиці виявилося, що умову $n_1 \geq n$ потрібно перевіряти програмно: сервер її не перевіряє, і при її порушенні успіх залежить від випадкового значення підпису.

Реалізація є навчальною: датчик `random` не є криптографічно стійким, а шифрування й підпис виконуються без доповнення та геш-функції. Для реального застосування цього недостатньо.

Додаток А. Кандидати, що не пройшли перевірку простоти

Кожен кандидат має вигляд $m_0 + 2i$. Для пробних ділень наведено запис «i (дільник)». Для тесту Міллера-Рабіна наведено сам кандидат; основи, що спростовують простоту, збережено у файлі `results/primes.json`.

p, m_0 = `852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD94D`

Пробні ділення (53): 0 (3), 1 (11), 2 (353), 3 (3), 4 (5), 5 (17), 6 (3), 7 (83), 9 (3), 10 (7), 12 (3), 13 (41), 14 (5), 15 (3), 16 (79), 17 (7), 18 (3), 19 (5), 21 (3), 22 (17), 23 (11), 24 (3), 26 (43), 27 (3), 28 (29), 29 (5), 30 (3), 31 (7), 32 (13), 33 (3), 34 (5), 35 (251), 36 (3), 37 (19), 38 (7), 39 (3), 42 (3), 44 (5), 45 (3), 47 (283), 48 (3), 49 (5), 51 (3), 52 (7), 53 (67), 54 (3), 56 (11), 57 (3), 58 (13), 59 (5), 60 (3), 63 (3), 64 (5).

i	Відкинуто тестом Міллера-Рабіна
8	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD95D`
11	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD963`
20	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD975`
25	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD97F`
40	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD99D`
41	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD99F`
43	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD9A3`
46	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD9A9`
50	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD9B1`
55	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD9BB`
61	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD9C7`
62	`852D78B7 9761D2B3 7FAFAD48 5EEDA1D8 52AE3F06 35840EA8 A1BEF5D3 BD9DD9C9`

q, m_0 = `88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 342988CD`

Пробні ділення (95): 0 (5), 1 (61), 2 (3), 4 (11), 5 (3), 6 (7), 7 (29), 8 (3), 10 (5), 11 (3), 12 (13), 13 (7), 14 (3), 15 (5), 16 (23), 17 (3), 18 (79), 19 (19), 20 (3), 22 (199), 23 (3), 24 (109), 25 (5), 26 (3), 27 (7), 28 (73), 29 (3), 30 (5), 31 (31), 32 (3), 34 (7), 35 (3), 36 (29), 37 (11), 38 (3), 39 (23), 40 (5), 41 (3), 43 (137), 44 (3), 45 (5), 46 (17), 47 (3), 48 (7), 50 (3), 51 (13), 52 (37), 53 (3), 54 (173), 55 (5), 56 (3), 57 (19), 59 (3), 60 (5), 61 (499), 62 (3), 63 (17), 64 (13), 65 (3), 66 (293), 68 (3), 69 (7), 70 (5), 71 (3), 73 (47), 74 (3), 75 (5), 76 (7), 77 (3), 79 (59), 80 (3), 81 (11), 83 (3), 85 (5), 86 (3), 88 (71), 89 (3), 90 (5), 91 (439), 92 (3), 93 (31), 94 (29), 95 (3), 96 (577), 97 (7), 98 (3), 99 (139), 100 (5), 101 (3), 103 (11), 104 (3), 105 (5), 106 (163), 107 (3), 108 (23).

i	Відкинуто тестом Міллера-Рабіна
3	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 342988D3`
9	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 342988DF`
21	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 342988F7`
33	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 3429890F`
42	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 34298921`
49	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 3429892F`
58	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 34298941`
67	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 34298953`
72	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 3429895D`
78	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 34298969`
82	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 34298971`
84	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 34298975`
87	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 3429897B`
102	`88DA8916 F12D135C 8DCCB14F 3097069A 1EB01B59 1234ED48 4E4E35C4 34298999`

$\mathbf{p_1}$, m_0 = `90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509C85`

Пробні ділення (190): 0 (3), 1 (11), 2 (7), 3 (3), 5 (59), 6 (3), 7 (131), 8 (5), 9 (3), 10 (263), 11 (647), 12 (3), 13 (5), 14 (31), 15 (3), 16 (7), 18 (3), 19 (19), 20 (29), 21 (3), 22 (43), 23 (5), 24 (3), 27 (3), 28 (5), 29 (17), 30 (3), 33 (3), 34 (11), 35 (887), 36 (3), 37 (7), 38 (5), 39 (3), 40 (431), 41 (127), 42 (3), 43 (5), 44 (7), 45 (3), 46 (17), 47 (13), 48 (3), 49 (29), 51 (3), 52 (227), 53 (5), 54 (3), 55 (89), 56 (11), 57 (3), 58 (5), 59 (41), 60 (3), 61 (223), 63 (3), 64 (59), 65 (7), 66 (3), 67 (11), 68 (5), 69 (3), 70 (23), 72 (3), 73 (5), 74 (569), 75 (3), 76 (19), 78 (3), 79 (7), 80 (17), 81 (3), 82 (439), 83 (5), 84 (3), 86 (7), 87 (3), 88 (5), 89 (11), 90 (3), 91 (233), 93 (3), 95 (19), 96 (3), 97 (17), 98 (5), 99 (3), 100 (7), 101 (379), 102 (3), 103 (5), 105 (3), 106 (421), 107 (7), 108 (3), 111 (3), 112 (13), 113 (5), 114 (3), 116 (23), 117 (3), 118 (5), 119 (251), 120 (3), 121 (7), 122 (11), 123 (3), 124 (61), 125 (13), 126 (3), 127 (73), 128 (5), 129 (3), 131 (17), 132 (3), 133 (5), 135 (3), 136 (29), 137 (179), 138 (3), 139 (23), 140 (499), 141 (3), 142 (7), 143 (5), 144 (3), 146 (211), 147 (3), 148 (5), 149 (7), 150 (3), 151 (13), 152 (19), 153 (3), 155 (11), 156 (3), 158 (5), 159 (3), 160 (197), 162 (3), 163 (5), 164 (13), 165 (3), 166 (11), 168 (3), 169 (31), 170 (7), 171 (3), 172 (79), 173 (5), 174 (3), 175 (137), 177 (3), 178 (5), 180 (3), 181 (113), 182 (17), 183 (3), 184 (7), 185 (23), 186 (3), 187 (173), 188 (5), 189 (3), 190 (13), 191 (7), 192 (3), 193 (5), 194 (29), 195 (3), 198 (3), 199 (11), 200 (31), 201 (3), 203 (5), 204 (3), 205 (7), 206 (37), 207 (3), 208 (5), 209 (19), 210 (3), 211 (139), 212 (7), 213 (3), 215 (449), 216 (3), 217 (839), 218 (5), 219 (3).

i	Відкинуто тестом Міллера-Рабіна
4	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509C8D`
17	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509CA7`
25	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509CB7`
26	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509CB9`
31	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509CC3`
32	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509CC5`
50	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509CE9`
62	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D01`
71	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D13`
77	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D1F`
85	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D2F`
92	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D3D`
94	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D41`
104	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D55`
109	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D5F`
110	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D61`
115	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D6B`
130	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D89`
134	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509D91`
145	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509DA7`
154	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509DB9`
157	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509DBF`
161	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509DC7`
167	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509DD3`
176	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509DE5`
179	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509DEB`
196	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E0D`
197	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E0F`
202	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E19`
214	`90C600CC 043B95E0 150AC7AF 9D103D6B 0557AA55 CAC31EC3 7BC8D445 1D509E31`

q_1, m_0 = `B0EC8513 0333A36A AFB930FC CF26DD01 03CD3BAB FEB950F5 81EA77F8 12AA4B6F`

Пробні ділення (33): 0 (3), 3 (3), 4 (19), 5 (5), 6 (3), 7 (7), 8 (61), 9 (3), 10 (5), 11 (11), 12 (3), 13 (919), 14 (7), 15 (3), 16 (13), 18 (3), 19 (109), 20 (5), 21 (3), 22 (11), 23 (19), 24 (3), 25 (5), 27 (3), 28 (7), 29 (13), 30 (3), 31 (163), 32 (307), 33 (3), 34 (23), 35 (5), 36 (3).

i	**Відкинуто тестом Міллера-Рабіна**
1	`B0EC8513 0333A36A AFB930FC CF26DD01 03CD3BAB FEB950F5 81EA77F8 12AA4B71`
2	`B0EC8513 0333A36A AFB930FC CF26DD01 03CD3BAB FEB950F5 81EA77F8 12AA4B73`
17	`B0EC8513 0333A36A AFB930FC CF26DD01 03CD3BAB FEB950F5 81EA77F8 12AA4B91`
26	`B0EC8513 0333A36A AFB930FC CF26DD01 03CD3BAB FEB950F5 81EA77F8 12AA4BA3`
37	`B0EC8513 0333A36A AFB930FC CF26DD01 03CD3BAB FEB950F5 81EA77F8 12AA4BB9`